

Folien-Analyse — CSS (Säule 2)

Quelle: [css/Day1 Syntax-Einbinden-Selektoren-Werte-Eigenschaften](#) · [Day2 Box-Modell-Anordnung-Sichtbarkeit-Variablen-Prioritäten-Flexbox-Grid-Media-Queries](#). Gegen MDN verifiziert. Folienfehler mit . Themen-IDs C1–C9.

CSS-Grundlagen (Querschnitt)

- **Regel:** `Selektor { Eigenschaft: Wert; }` — Selektor (wer), Property (was), Value (wie).
- **3 Einbindungen:** (1) Inline `style="..."` (Selektor entfällt), (2) `<style>`-Tag im Head, (3) externe Datei `<link rel="stylesheet" href="...">` (seitenübergreifend).
- **3 Stylesheet-Quellen:** Browser-Default < Autor/Homepage < Benutzer.
- **Vererbung:** FOLIE sagt "fast alle Eigenschaften werden vererbt, Ausnahme Positionierung". Korrekt (MDN): NUR bestimmte (meist textbezogene) Eigenschaften erben (`color`, `font-*`, `line-height`); **Box-/Layout-Eigenschaften** (`margin`, `padding`, `border`, `background`, `width`, **Positionierung**) erben NICHT.

C1 — Selektoren & Matching (Tracing/MC, hoch)

Einfache Selektoren

Einfache Selektoren wählen Elemente über Typ, Klasse oder ID aus - die häufigsten Bausteine jeder Regel:

Selektor	Matcht	Stolperfalle
*	alle Elemente	Universalselektor
X (Typ)	alle Tags vom Typ X	z.B. a
.Y (Klasse)	Elemente mit Klasse Y	matcht auch <code>class="a Y b"</code>
#Z (ID)	genau das Element mit id Z	ID einmalig

Attribut-Selektoren

Attribut-Selektoren greifen Elemente anhand ihrer Attribute; die Operatoren unterscheiden exakte, Wort- und Teilstring-Treffer:

Selektor	Matcht	Beispiel
[a]	Attribut a gesetzt	<code>*[onclick]</code>
[a=wert]	EXAKT gleich	<code>class="awe"</code> ja, <code>"awe x"</code> nein
[a~=wert]	wert als WORT in Liste	<code>class="ab xy cd"</code> matcht <code>[class~=xy]</code> ; <code>[class~=x]</code> NICHT
[a\ =wert]	exakt wert ODER wert-...	<code>lang="en-us"</code> matcht <code>[lang\ =en]</code>
[a^=wert]	beginnt mit wert	<code>id="abxy"</code> → <code>[id^=ab]</code>
[a\$=wert]	endet mit wert	<code>[id\$=en]</code>
[a*=wert]	enthält wert	<code>[id*=bcd]</code>
[s1][s2]	beide Bedingungen	<code>input[lang=de][name*=xy]</code>

Kombinatoren

Kombinatoren beschreiben die Beziehung zwischen zwei Elementen - Verschachtelung oder Nachbarschaft:

Selektor	Bedeutung	Folien-Name (nicht-Standard)
E F	F ist Nachfahre von E (beliebig tief)	Descendant
E > F	F direktes Kind von E	Child
E ~ F	alle nachfolgenden Geschwister	Folie "Twin" → MDN: general sibling

Selektor	Bedeutung	Folien-Name (nicht-Standard)
E + F	direkt folgendes Geschwister	Folie "Neighbor" → MDN: adjacent/next sibling

Pseudoklassen / -elemente

Pseudoklassen wählen Elemente nach Zustand oder Position, Pseudoelemente sprechen Teile eines Elements an:

- Zustand: `:link`, `:visited`, `:hover`, `:active`, `:focus`, `:enabled`, `:disabled`, `:checked`, `:target`, `:lang(x)`.
- Strukturell: `:first-child`, `:last-child`, `:nth-child(n)` (n = Zahl, `odd`, `even`, `An+B`), `:only-child`, `:first-of-type`, `:nth-of-type` ...
- Pseudo-Elemente (doppelter Doppelpunkt): `::marker`, `::before`, `::after`, `::first-letter`, `::first-line`.

Fallen / Bugs

- **Falle (Leerzeichen!):** `a:link` = alle nicht-besuchten Links. `a :link` = Links INNERHALB eines `a` (Descendant!). `:link = *:link`.
- **Falle:** `ul:first-child` matcht nur, wenn `ul` selbst erstes Kind ist — `ul :first-child` (mit Space) matcht erstes Kind INNERHALB `ul`.
- **Falle:** `[class~=x]` (Wort) $\neq$ `[class=x]` (exakt) $\neq$ `[class*=x]` (Teilstring).
- **Falle:** Pseudoklasse direkt anhängen, OHNE Leerzeichen (`input:focus`, nicht `input :focus`).

Drill

- [Tracen] `<p class="awe maria">`. Welche matchen: `.awe`, `p[class="awe"]`, `[class~=awe]`, `[class*=aw]` ?

Lösung: `.awe`, `p[class="awe"]` (nicht exakt), `[class~=awe]`, `[class*=aw]`.

- [MC] `div p` vs `div>p`: Unterschied?

Lösung: `div p` = jeder `p` irgendwo unter `div`; `div>p` = nur direkte Kind- `p`.

C2 — Spezifität & Kaskade (Tracing, hoch)

Priorisierung (Reihenfolge)

Bei konkurrierenden Regeln entscheidet die Kaskade in dieser Rangfolge, welche gewinnt:

1. **Ursprung & Wichtigkeit:** important-Benutzer > important-Autor > Autor > Benutzer > Browser.
 2. **Spezifität** (von hoch nach niedrig): - Inline-Style (`style="..."`) - ID-Selektor `#x` - Klasse `.x` / Attribut `[...]` / Pseudoklasse `:hover` - Typ `h1` / Pseudo-Element `::marker`
 3. **Reihenfolge:** bei Gleichstand gewinnt die zuletzt notierte Regel.
- `!important` überschreibt normale Regeln: `Selektor { prop: wert !important; }` — laut Folie in App-Entwicklung Ausnahme, in Frameworks verboten.

Spezifität rechnen (MDN-Modell, a,b,c)

- a = #IDs, b = .Klassen/[Attr]/:pseudo-class, c = Typen/:pseudo-element. Inline schlägt alles, `!important` darüber. Universalselektor `*` zählt 0.

Fallen / Bugs

- **Falle:** ID (1 ID) schlägt beliebig viele Klassen (`#a` > `.x.y.z`).
- **Falle:** `!important` schlägt höhere Spezifität — typische Trace-Falle.
- **Falle:** Bei gleicher Spezifität entscheidet Quellreihenfolge (später gewinnt).

Drill

- [Tracen] `p#a{color:red}` vs `.c{color:blue !important}` auf `<p id="a" class="c">` ?

Lösung: blau — `!important` schlägt ID-Spezifität.

- [Tracen] Welcher gewinnt: `#x` oder `.a.b.c` ?

Lösung: `#x` (eine ID > drei Klassen).

C3 — Box-Modell (Tracing/MC, hoch)

- **Schichten (innen→außen):** content → padding → border → margin .

Eigenschaft	Funktion
padding(-top/right/bottom/left)	Innenabstand
margin(-...)	Außenabstand
border-width/-style/-color	Rahmen, Kurzform border: 1px solid red;
width/height (+min/-max-)	Größe (Standard = nur content-Box)
overflow	visible (ragt raus), hidden (abgeschnitten), scroll, auto

- **Shorthand 4 Werte:** padding: 10px 20px 30px 40px; = top right bottom left (im Uhrzeigersinn). 1 Wert = alle gleich.
- **border-style-Werte:** solid, dotted, dashed, double, groove, ridge, inset, outset, hidden, none .
- **border-collapse** (Tabellen): separate / collapse . Bei collapse : none +border → border gewinnt; hidden +border → hidden unterdrückt.
- **ERGÄNZUNG** (nicht auf Folie, aber Tracker C3): box-sizing: content-box (Default, width = nur Inhalt) vs. border-box (width inkl. padding+border). Klausurrelevant für Breitenrechnungen.

Fallen / Bugs

- **Falle:** Standard box-sizing: content-box → Gesamtbreite = width + padding + border . Mit border-box zählt padding/ border in width rein.
- **Falle:** margin kollabiert vertikal zwischen benachbarten Blöcken (größerer Wert gewinnt, nicht Summe).

Drill

- [Tracen] width:100px; padding:10px; border:5px; box-sizing:content-box . Gesamtbreite?

Lösung: $100 + 2 \cdot 10 + 2 \cdot 5 = 130\text{px}$.

- [Tracen] Gleiche Werte mit box-sizing:border-box . Content-Breite?

Lösung: $100 - 20 - 10 = 70\text{px}$ content, Gesamtbreite 100px.

C4 — Positionierung (MC/Tracing, hoch)

position	Bezug	Platz im Fluss
static	normaler Fluss (Default)	ja
relative	Verschiebung relativ zur eigenen Anfangsposition	behält Platz
absolute	nächster Vorfahr mit position≠static (sonst Viewport)	gibt Platz frei , scrollt mit
fixed	Browserfenster	gibt Platz frei , scrollt NICHT mit
sticky	nächster Scroll-Container	toggelt relative/fixed

- Offsets: top/right/bottom/left . Überlagerung: z-index (größer = vorne).
- **Sichtbarkeit:** visibility: hidden (unsichtbar, **belegt Platz**) vs display: none (unsichtbar, **kein Platz**) vs visibility: collapse (wie hidden, bei <tr> Platz frei).
- **float:** left/right/none (Element wird umflossen); clear: left/right/both/none bricht Umfluss ab.
- **display:** block, inline, inline-block, list-item, none, flex, grid... .

Fallen / Bugs

- **Kontrast:** `absolute` bezieht sich auf nächsten NICHT-static Vorfahren; `fixed` immer aufs Fenster.
- **Kontrast:** `visibility:hidden` belegt Platz, `display:none` nicht. **Häufige MC-Falle.**
- **Falle:** `absolute` / `fixed` nehmen Element aus dem Fluss → folgende Elemente rücken auf.

Drill

- [MC] Element soll beim Scrollen am Platz bleiben: `fixed` oder `absolute` ?

Lösung: `fixed` .

- [MC] Unsichtbar, aber Platz bleibt: `display:none` oder `visibility:hidden` ?

Lösung: `visibility:hidden` .

C5 — Display & Flow

- `block` (eigene Zeile), `inline` (im Textfluss, width/height ignoriert), `inline-block` (inline angeordnet, aber width/height/Box wirken), `none` (weg).
- `<span>` = generisches Inline, `<div>` = generisches Block (Gruppierung).

Drill

- [MC] Welcher display erlaubt width/height UND inline-Anordnung?

Lösung: `inline-block` .

C6 — Einheiten

Einheit	Bezug
px	absolute Pixel
em	Schriftgröße des (Eltern-)Elements
rem	Schriftgröße des Root (html)
%	relativ zum Elternelement
vh/vw	Viewport-Höhe/-Breite
fr	Grid-Anteil (fraction)
cm/mm/in	physische Maße (Druck)
* Farben: Schlüsselwörter (red), #RRGGBB hex.	

Drill

- [MC] `2em` bei Eltern-font-size 16px = ? / `2rem` bei root 10px = ?

Lösung: 32px / 20px.

C7 — Flexbox & Grid (Produktion, hoch)

Flexbox (1-dimensional)

Flexbox ordnet Elemente entlang EINER Achse an und eignet sich für Zeilen- oder Spalten-Layouts:

- Container: `display: flex;` . Direkte Kinder = Flex-Items. | Eigenschaft (Container) | Funktion | | :--- | :--- | | `flex-direction` | `row` / `row-reverse` / `column` / `column-reverse` | | `flex-wrap` | `nowrap` / `wrap` / `wrap-reverse` | | `flex-flow` | Kurzform

`<direction> <wrap>` || `justify-content` | Hauptachse: `flex-start` / `flex-end` / `center` / `space-between` / `space-around` || `align-items` | Querachse: `flex-start` / `flex-end` / `center` / `baseline` || `gap` / `row-gap` / `column-gap` | Abstände |

Eigenschaft (Item)	Funktion
<code>order</code>	Reihenfolge (ganze Zahl, kleiner = vorne)
<code>flex-grow</code>	Wachstumsfaktor
<code>flex-shrink</code>	Schrumpffaktor
<code>flex-basis</code>	Basisgröße (FOLIE schreibt "flex-start" — gemeint ist flex-basis)
<code>flex</code>	Kurzform <code><grow> <shrink> <basis></code>

Grid (2-dimensional)

Grid legt ein zweidimensionales Raster aus Zeilen und Spalten an - für komplexe Seitenlayouts:

- Container: `display: grid;` | Eigenschaft | Funktion || `:-` | `:-` || `grid-template-columns` / `-rows` | Spalten/Zeilen, Anzahl Werte = Anzahl Spalten/Zeilen; Einheit `fr`, `auto` || `repeat(n, ...)` | Wiederholung, `auto-fill`, `minmax(min,max)` || `grid-column` / `grid-row` | `<start>/<end>`, negativ vom Ende (`1/-1` = ganze Zeile) || `grid-area` | `row-start/col-start/row-end/col-end` || `gap` | Abstände || `justify-items` / `align-items`, `justify-self` / `align-self` | Item-Ausrichtung |

Flexbox vs. Grid

- Flexbox = **1D** (Zeile ODER Spalte), Größe auf Items. Grid = **2D** (Zeilen UND Spalten), Größe auf Container. Faustregel: Seitenlayout=Grid, Komponente=Flexbox.

Fallen / Bugs

- **Falle:** `justify-content` = Hauptachse, `align-items` = Querachse — bei `flex-direction:column` vertauschen sich die Achsen!
- **Falle:** `fr` bestimmt Anteil am freien Platz, nicht die Inhaltsbreite.

Drill

- [Erstellen] 3 Items horizontal zentriert (Haupt- + Querachse).

Lösung: `.c{display:flex; justify-content:center; align-items:center;}`

- [Erstellen] Grid mit 3 gleich breiten Spalten.

Lösung: `display:grid; grid-template-columns: repeat(3, 1fr);`

C8 — Media Queries / Responsive

- `@media <typ> and (feature) { ... }`. Typen: `all`, `print`, `screen`. Features: `width`, `min-width`, `max-width`, `orientation`, `prefers-color-scheme` ...
- Logik: `and` (UND), `,` (ODER), `not` (Negierung).

```
@media print { nav { display: none; } }
@media screen and (min-width: 1000px) { #content { max-width: 80%; margin: auto; } }
```

Drill

- [Erstellen] Regel nur für Bildschirme breiter als 600px.

Lösung: `@media screen and (min-width:600px){ ... }`

C9 — CSS-Variablen (Custom Properties)

- Deklarieren: `:root { --main-bg: red; }`. Lesen: `var(--main-bg)`. Fallback: `var(--x, #FF0000)`.

- **Wichtig:** Variable speichert keinen festen Wert — wird zum Nutzungszeitpunkt berechnet; vererbt sich an Kinder; zur Laufzeit überschreibbar.
- Custom Properties beginnen mit `--`, sind case-sensitiv.

Fallen / Bugs

- **Falle:** Ungültiger var-Wert ohne Fallback → Eigenschaft nutzt geerbten/Initialwert (nicht "rot wie root", sondern Vererbungskette).

Drill

- [Tracen] `div{background:var(--c)}, .two{--c:blue}`. `<div class="two"><div class="four"></div></div>` — Farbe von `.four` ?

Lösung: blau (geerbt von `.two`).